Register Number:192425325
Name:B.Archana

CO5-AT-1

MLA0201-Fundamentals Of Machine Learning
Constraint-Based Problem Solving

Question 1: Reinforcement Learning for Warehouse Robot Navigation

Proposed Reinforcement Learning Solution

For the warehouse robot, we can model the problem as a **Markov Decision Process (MDP)**.

- **State (S):** Robot's current grid position and remaining battery/moves.
- **Actions (A):** Up, Down, Left, Right.
- **Reward (R):**
  - +100 → Reaching the destination
  - +5 → Moving through a good path
  - −10 → Attempting to move into an obstacle
  - −1 → Normal movement (to encourage shorter paths)
  - −2 → Additional battery/movement cost
- **Transition:** Because the environment is stochastic, the intended action succeeds with high probability, while the robot may move in another direction with a smaller probability.
- **Discount factor:** $\gamma = 0.9$.
- **Maximum moves:** 25.
- **Battery capacity:** 30 units.

Exploration Strategy

I would select an **ε-greedy exploration strategy**.

With probability ε, the robot chooses a random action to explore the warehouse. Otherwise, it selects the action with the highest estimated value.

For example:

$\varepsilon = 0.10$

10% → Explore a random action
90% → Choose the best known action

This is suitable because the robot needs to **explore alternative routes** while still preferring routes that are short, safe, and energy-efficient.

Value Iteration Approach

Value Iteration is appropriate because the warehouse environment has:

- A finite number of states.
- Four possible actions.
- Obstacles and rewards.
- Stochastic transitions.
- A clear objective of reaching the destination.

The Bellman optimality equation is:

$$V(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P\,(s', \mid sa) V(s') \right]$$

After calculating the optimal value function, the policy is:

Code:

```
import numpy as np
import random

warehouse=[
['S','.','.','#','.'],
['.','#','.','#','.'],
['.','#','.','.','.'],
['.','.','#','#','.'],
['#','.','.','.','G']
]

ROWS=len(warehouse)
COLS=len(warehouse[0])
START=(0,0)
GOAL=(4,4)

actions={
'Up':(-1,0),
'Down':(1,0),
'Left':(0,-1),
'Right':(0,1)
}
action_list=list(actions.keys())

GAMMA=0.90
THETA=0.001
EPSILON=0.10
MAX_MOVES=25
BATTERY=30

def valid_state(position):
    r,c=position
    if r<0 or r>=ROWS or c<0 or c>=COLS:
        return False
    if warehouse[r][c]=='#':
        return False
    return True

def move(state,action):
    r,c=state
    dr,dc=actions[action]
    new_state=(r+dr,c+dc)
    if valid_state(new_state):
        return new_state
    return state

def left_action(action):
    order=['Up','Right','Down','Left']
    i=order.index(action)
    return order[(i-1)%4]
```

```
def right_action(action):
    order=['Up','Right','Down','Left']
    i=order.index(action)
    return order[(i+1)%4]

def transitions(state,action):
    result=[]
    result.append((0.80,move(state,action)))
    result.append((0.10,move(state,left_action(action))))
    result.append((0.10,move(state,right_action(action))))
    return result

def get_reward(state,action,next_state):
    if next_state==GOAL:
        return 100
    if next_state==state:
        return -10
    return -2

states=[]
for r in range(ROWS):
    for c in range(COLS):
        if warehouse[r][c]!='#':
            states.append((r,c))

V={state:0.0 for state in states}
V[GOAL]=100.0

iteration=0

while True:
    delta=0
    new_V=V.copy()
    for state in states:
        if state==GOAL:
            continue
        values=[]
        for action in action_list:
            expected_value=0
            for probability,next_state in transitions(state,action):
                reward=get_reward(state,action,next_state)
                expected_value+=probability*(reward+GAMMA*V[next_state])
            values.append(expected_value)
        best_value=max(values)
        new_V[state]=best_value
        delta=max(delta,abs(V[state]-best_value))
    V=new_V
    iteration+=1
    if delta<THETA:
        break
  policy={}

  for state in states:
      if state==GOAL:
          policy[state]='Goal'
          continue
      best_action=None
      best_value=-float('inf')
      for action in action_list:
          expected_value=0
          for probability,next_state in transitions(state,action):
              reward=get_reward(state,action,next_state)
              expected_value+=probability*(reward+GAMMA*V[next_state])
          if expected_value>best_value:
              best_value=expected_value
              best_action=action
      policy[state]=best_action

  print("="*50)
  print("VALUE ITERATION RESULTS")
  print("="*50)
  print("Iterations:",iteration)
  print("\nVALUE FUNCTION")
```

```
for r in range(ROWS):
    row=[]
    for c in range(COLS):
        if warehouse[r][c]=='#':
            row.append("#####")
        else:
            row.append(f"{V[(r,c)]:5.1f}")
    print(" ".join(row))

symbols={
'Up':'↑',
'Down':'↓',
'Left':'←',
'Right':'→',
'Goal':'G'
}

print("\nOPTIMAL POLICY")

for r in range(ROWS):
    row=[]
    for c in range(COLS):
        if warehouse[r][c]=='#':
            row.append('#')
        else:
            row.append(symbols[policy[(r,c)]])
    print(" ".join(row))

def epsilon_greedy(state):
    if random.random()<EPSILON:
        return random.choice(action_list)
    return policy[state]

print("\n"+"="*50)
print("ROBOT SIMULATION")
print("="*50)

current_state=START
battery=BATTERY
moves=0
path=[current_state]

print("Starting position:",current_state)
print("Initial battery:",battery)

while current_state!=GOAL and moves<MAX_MOVES:
    action=epsilon_greedy(current_state)
    possible_transitions=transitions(current_state,action)
    probabilities=[p for p,s in possible_transitions]
    index=np.random.choice(len(possible_transitions),p=probabilities)
    next_state=possible_transitions[index][1]

    battery-=1
    moves+=1
    print(f"Move {moves}: {current_state} -> {next_state} | Action: {action} | Battery: {battery}"
    current_state=next_state
    path.append(current_state)

print("\n"+"="*50)
print("FINAL RESULT")
print("="*50)

if current_state==GOAL:
    print("Destination reached successfully!")
    print("Total moves:",moves)
    print("Battery used:",BATTERY-battery)
    print("Remaining battery:",battery)
elif moves>=MAX_MOVES:
    print("Maximum 25 moves reached.")
    print("Total moves:",moves)
    print("Remaining battery:",battery)
else:
    print("Robot stopped.")
    print("Remaining battery:",battery)

print("\nRobot Path:")
print(path)

print("\nEXPLORATION STRATEGY: Epsilon-Greedy")
print("EPSILON:",EPSILON)
print("ALGORITHM: Value Iteration")
print("DISCOUNT FACTOR:",GAMMA)
print("MAXIMUM MOVES:",MAX_MOVES)
print("BATTERY CAPACITY:",BATTERY)
print("ENVIRONMENT: Stochastic")
```

Output:

```
...
==================================================
VALUE ITERATION RESULTS
==================================================
Iterations: 25

VALUE FUNCTION
 52.4  62.4  76.0 ##### 111.0
 62.2 #####  92.2 ##### 131.4
 75.9 ##### 110.0 131.4 154.7
 91.4 109.9 ##### ##### 182.9
##### 131.4 156.2 182.9 100.0

OPTIMAL POLICY
→ → ↓ # ↓
↓ # ↓ # ↓
↓ # → → ↓
→ ↓ # # ↓
# → → → G

==================================================
ROBOT SIMULATION
==================================================
Starting position: (0, 0)
Initial battery: 30
Move 1: (0, 0) -> (0, 1) | Action: Right | Battery: 29
Move 2: (0, 1) -> (0, 2) | Action: Right | Battery: 28
Move 3: (0, 2) -> (1, 2) | Action: Down | Battery: 27
Move 4: (1, 2) -> (2, 2) | Action: Down | Battery: 26
Move 5: (2, 2) -> (2, 3) | Action: Right | Battery: 25
Move 6: (2, 3) -> (2, 4) | Action: Right | Battery: 24
Move 7: (2, 4) -> (2, 4) | Action: Down | Battery: 23
Move 8: (2, 4) -> (2, 3) | Action: Left | Battery: 22
Move 9: (2, 3) -> (2, 4) | Action: Right | Battery: 21
Move 10: (2, 4) -> (2, 3) | Action: Down | Battery: 20
Move 11: (2, 3) -> (2, 4) | Action: Right | Battery: 19
Move 12: (2, 4) -> (3, 4) | Action: Down | Battery: 18
Move 13: (3, 4) -> (4, 4) | Action: Down | Battery: 17

==================================================
FINAL RESULT
==================================================
Destination reached successfully!
Total moves: 13
Battery used: 13
Remaining battery: 17

Robot Path:
[(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (2, 4), (2, 3), (2, 4), (2, 3), (2, 4), (3, 4), (4, 4)]

EXPLORATION STRATEGY: Epsilon-Greedy
EPSILON: 0.1
ALGORITHM: Value Iteration
DISCOUNT FACTOR: 0.9
MAXIMUM MOVES: 25
BATTERY CAPACITY: 30
ENVIRONMENT: Stochastic
```

Why Value Iteration is Suitable

Value Iteration is selected instead of Policy Iteration because it directly calculates the optimal value of each state and then derives the policy.

It is particularly suitable here because:

1. **Small state/action space** – The warehouse has a manageable number of grid states.
2. **Stochastic environment** – Value Iteration incorporates transition probabilities.
3. **Obstacle avoidance** – Invalid movements receive negative rewards.
4. **Battery conservation** – Every movement has a cost.
5. **Time constraint** – Movement penalties encourage shorter routes.
6. **Goal-oriented** – Reaching the destination receives a large positive reward.
7. **Exploration** – ε-greedy exploration allows the robot to discover alternative routes.

Final Answer:

Exploration strategy: ε-greedy
Planning algorithm: Value Iteration
State: Robot position
Actions: Up, Down, Left, Right
Goal reward: +100

Movement cost: −2
Obstacle penalty: −10
Discount factor: $\gamma = 0.9$
Maximum moves: 25
Battery: 30 units
Environment: Stochastic
Therefore, **ε-greedy exploration combined with Value Iteration** is an appropriate Reinforcement Learning solution for finding a safe, short, and energy-efficient warehouse navigation policy.

Question 2 — Multi-Armed Bandit for Online Advertisement Selection

Problem Formulation
This problem can be formulated as a **K-Armed Bandit problem** where:

- **K = 5** advertisements.
- Each advertisement is an **arm**.
- One advertisement is displayed to each visitor.
- A **click = reward 1**.
- No click = **reward 0**.
- The click probability of each advertisement is initially unknown.
- Total available interactions = **10,000**.
- Objective = **maximize total clicks and minimize cumulative regret**.
-

Mathematical formulation
Let the five advertisements be:

$$A = \{A_1, A_2, A_3, A_4, A_5\}$$

Each advertisement has an unknown click probability:

$$\mu_1, \mu_2, \mu_3, \mu_4, \mu_5$$

At each interaction t, the algorithm selects one advertisement a_t.
The reward is:

$$R_t = \begin{cases} 1 & \text{if the user clicks the advertisement} \\ 0 & \text{otherwise} \end{cases}$$

The objective is:

$$\max \sum 10000\, R_t$$

while minimizing cumulative regret.

Selected Exploration Strategy — UCB
I select the **Upper Confidence Bound (UCB)** strategy.
The UCB formula is:

$$UCB_i = \bar{X}_i + \sqrt{\frac{2\ln(t)}{N_i}}$$

where:

- $\bar{X}_i$= average reward/estimated CTR of advertisement i
- N_i= number of times advertisement $i$has been displayed
- t= total number of interactions.

The first term represents **exploitation** and the second term represents **exploration**.

Why UCB?

UCB is suitable because:

- It learns unknown click probabilities.
- It explores advertisements that have been tried fewer times.
- It increasingly exploits advertisements with high CTR.
- It does not require manually choosing a fixed exploration percentage.
- It is appropriate when the interaction budget is limited to 10,000.
- It helps reduce cumulative regret.

Reward Mechanism

The reward is directly related to advertisement performance.
User clicks advertisement → Reward = 1
User does not click → Reward = 0
For example:
Advertisement 1 → CTR = 5%
Advertisement 2 → CTR = 10%
Advertisement 3 → CTR = 15%
Advertisement 4 → CTR = 8%
Advertisement 5 → CTR = 20%
The algorithm does **not initially know** these probabilities.
As more visitors interact with the advertisements, the algorithm estimates their CTRs.
An advertisement receiving more clicks gets a higher estimated reward and is therefore selected more frequently.

Code:

```
import numpy as np
import random
NUM_ADS=5
TOTAL_INTERACTIONS=10000
true_probabilities=[0.05,0.10,0.15,0.08,0.20]
counts=np.zeros(NUM_ADS)
rewards=np.zeros(NUM_ADS)
total_reward=0
cumulative_regret=0
ad_history=[]
reward_history=[]
regret_history=[]
for t in range(1,TOTAL_INTERACTIONS+1):
    if t<=NUM_ADS:
        ad=t-1
    else:
```

```
            ucb_values=[]
            for i in range(NUM_ADS):
                average_reward=rewards[i]/counts[i]
                exploration=np.sqrt((2*np.log(t))/counts[i])
                ucb_value=average_reward+exploration
                ucb_values.append(ucb_value)
            ad=np.argmax(ucb_values)
        click=np.random.binomial(1,true_probabilities[ad])
        counts[ad]+=1
        rewards[ad]+=click
        total_reward+=click
        best_probability=max(true_probabilities)
        regret=best_probability-true_probabilities[ad]
        cumulative_regret+=regret
        ad_history.append(ad)
        reward_history.append(click)
        regret_history.append(cumulative_regret)
print("="*60)
print("MULTI-ARMED BANDIT FOR ONLINE ADVERTISEMENT")
print("="*60)
print("Number of Advertisements:",NUM_ADS)
print("Total Interactions:",TOTAL_INTERACTIONS)
print("Exploration Strategy: UCB")
print()
print("="*60)
print("ADVERTISEMENT PERFORMANCE")
print("="*60)
for i in range(NUM_ADS):
    estimated_ctr=rewards[i]/counts[i]
    print(f"Advertisement {i+1}: Interactions={int(counts[i])}, Clicks={int
estimated_ctrs=rewards/counts
best_ad=np.argmax(estimated_ctrs)
print()
print("="*60)
print("FINAL RESULT")
print("="*60)
print("Best Advertisement:",best_ad+1)
print("Estimated CTR:",f"{estimated_ctrs[best_ad]:.4f}")
print("Total Clicks:",int(total_reward))
print("Total Interactions:",TOTAL_INTERACTIONS)
print("Overall CTR:",f"{total_reward/TOTAL_INTERACTIONS:.4f}")
print("Cumulative Regret:",f"{cumulative_regret:.4f}")
print()
print("="*60)
print("REWARD MECHANISM")
print("="*60)
print("Click = Reward 1")
print("No Click = Reward 0")
print()
print("Advertisements with higher estimated CTR are selected more frequentl
print()
```

```
print("="*60)
print("ADVERTISEMENT SELECTION FREQUENCY")
print("="*60)
for i in range(NUM_ADS):
    percentage=(counts[i]/TOTAL_INTERACTIONS)*100
    print(f"Advertisement {i+1}: {int(counts[i])} selections ({percentage:.2f}%)")
```

Output:

```
============================================================
MULTI-ARMED BANDIT FOR ONLINE ADVERTISEMENT
============================================================
Number of Advertisements: 5
Total Interactions: 10000
Exploration Strategy: UCB

============================================================
ADVERTISEMENT PERFORMANCE
============================================================
Advertisement 1: Interactions=456, Clicks=22, Estimated CTR=0.0482
Advertisement 2: Interactions=841, Clicks=85, Estimated CTR=0.1011
Advertisement 3: Interactions=2079, Clicks=322, Estimated CTR=0.1549
Advertisement 4: Interactions=617, Clicks=47, Estimated CTR=0.0762
Advertisement 5: Interactions=6007, Clicks=1174, Estimated CTR=0.1954

============================================================
FINAL RESULT
============================================================
Best Advertisement: 5
Estimated CTR: 0.1954
Total Clicks: 1650
Total Interactions: 10000
Overall CTR: 0.1650
Cumulative Regret: 330.4900

============================================================
REWARD MECHANISM
============================================================
Click = Reward 1
No Click = Reward 0

Advertisements with higher estimated CTR are selected more frequently by the UCB algorithm.

============================================================
ADVERTISEMENT SELECTION FREQUENCY
============================================================
Advertisement 1: 456 selections (4.56%)
Advertisement 2: 841 selections (8.41%)
Advertisement 3: 2079 selections (20.79%)
Advertisement 4: 617 selections (6.17%)
Advertisement 5: 6007 selections (60.07%)
```

Exploration vs Exploitation

UCB automatically balances both requirements.

Exploration

The exploration term:

$$\sqrt{\frac{2\ln(t)}{N_i}}$$

becomes larger when an advertisement has been selected fewer times.

Therefore, advertisements with insufficient information are given opportunities to be tested.

Exploitation

The estimated reward:

$$\bar{X}_i$$

becomes important as more data are collected.

Advertisements with higher estimated CTR receive higher UCB values and are selected more frequently.
Thus:

Early stage
↓
Explore all 5 advertisements
↓
Collect click information
↓
Estimate CTR
↓
Identify high-performing advertisements
↓
Select high-CTR advertisements more frequently
↓
Maximize clicks

Cumulative Regret

Regret measures how much reward is lost by selecting an advertisement that is not the best advertisement.
The instantaneous regret is:

$$r_t = \mu^* - \mu_{a_t}$$

where:

- μ^*= highest click probability
- μ_{a_t}= probability of the selected advertisement.

Cumulative regret is:

$$R_T = \sum_{t=1}^{T} (\mu^* - \mu_{a_t})$$

For this problem:

$$T = 10000$$

Therefore, the algorithm attempts to learn the best advertisement quickly so that fewer interactions are wasted on inferior advertisements.

Why UCB Satisfies the Given Constraints:

Constraint	**UCB Solution**
Five advertisements	Treats each advertisement as an arm
One advertisement per visitor	Selects exactly one arm at each interaction
Unknown click probability	Learns CTR from observed rewards
Exploration required	Confidence-bound term encourages exploration
Exploitation required	High estimated CTR advertisements are preferred
10,000 interactions	Uses the available interactions efficiently
Maximize rewards	Selects advertisements with high expected reward
Minimize regret	Reduces unnecessary selection of poor advertisements

Conclusion

The online advertisement problem is formulated as a 5-Armed Bandit problem, where each advertisement represents an arm and a user click represents a reward of 1. Since the click probabilities are initially unknown, the system must learn them through interaction.

The UCB algorithm is selected because it effectively balances exploration and exploitation. It explores advertisements with limited observations while exploiting advertisements with high estimated CTR. The reward mechanism directly influences advertisement selection because advertisements receiving more clicks obtain higher estimated rewards and are consequently selected more often.

With a maximum of 10,000 user interactions, UCB aims to maximize total clicks and minimize cumulative regret, making it an appropriate strategy for online advertisement selection.

Question 3: Sampling Strategy for Machine Learning Model Development

Introduction

The healthcare company has a dataset containing **1,000,000 patient records**, but only **8 GB RAM** is available for training. Therefore, using the complete dataset may require excessive computational resources.

The main objectives are:

- Reduce computational cost.
- Maintain representation of all patient categories.
- Minimize sampling bias.
- Achieve **more than 95% prediction accuracy**.
- Maintain **at least 95% confidence**.
- Satisfy sample-complexity and generalization requirements.

Recommended Method: Stratified Random Sampling

I recommend **Stratified Random Sampling** because it ensures that important patient categories are represented in approximately the same proportions as in the original population.

2. Stratified Random Sampling

In stratified sampling, the complete dataset is divided into groups called **strata** based on an important characteristic.

For a disease-prediction dataset, strata could be:

- Disease category
- Age group
- Risk level
- Other clinically relevant categories

A random sample is then selected from each group.

Example

Suppose the original dataset has:

Patient Category	Population
Category A	60%
Category B	25%
Category C	10%
Category D	5%

If we select 10,000 records, approximately:

Patient Category	Sample
Category A	6,000

Category B	2,500
Category C	1,000
Category D	500

Thus, the sample remains representative.

Why Stratified Sampling is Best

Simple Random Sampling

Every patient has an equal probability of being selected.

Problem: Rare disease categories may be poorly represented in the sample.

Stratified Random Sampling

Patients are divided into categories and sampled from each category.

Advantage: Important and minority categories are preserved.

Monte Carlo Sampling

It repeatedly uses random sampling/simulation to estimate outcomes.

Problem: It is not the most direct solution when the primary requirement is to preserve patient-category representation.

Therefore:

Stratified Random Sampling is the most suitable method.

Sample Complexity:

Sample complexity refers to the amount of training data required for a machine learning model to learn a reliable relationship and generalize to unseen data.

A commonly used statistical sample-size formula is:

$$n = \frac{Z^2 p(1-p)}{E^2}$$

Where:

- n= required sample size
- Z= confidence value
- p= estimated population proportion
- E= acceptable error

For 95% confidence:

$$Z = 1.96$$

Using the conservative assumption:

$$p = 0.5$$

and:

$$E = 0.01$$

we obtain:

$$n = \frac{(1.96)^2(0.5)(0.5)}{(0.01)^2}$$
$$n \approx 9604$$

Therefore, approximately **9,604 observations** are required for this basic proportion-estimation setting.

However, this calculation **does not guarantee 95% ML classification accuracy**. The final sample size should be verified using validation performance and learning curves.

VC Dimension:

VC Dimension (Vapnik–Chervonenkis Dimension) represents the capacity or complexity of a learning model.

A model with a high VC dimension can learn complex patterns, but it generally requires more training data to generalize reliably.

A simplified relationship is:

$$m = O\left(\frac{d + \ln(1/\delta)}{\epsilon}\right)$$

Where:

- m= required number of samples
- d= VC dimension
- ϵ= acceptable error
- δ= probability of failure

For 95% confidence:

$$\delta = 0.05$$

Therefore:

$$1 - \delta = 0.95$$

Effect on this problem

A highly complex model may require more training samples and more memory. Therefore, an appropriately sized model should be selected so that it can learn the disease patterns without unnecessary complexity.

Occam's Learning Principle

Occam's Learning Principle states that when multiple models perform similarly, the simpler model is generally preferred.

In this problem:

Complex Model
↓
Higher computational cost
↓
More samples required
↓
Higher risk of overfitting

Whereas:

Appropriate Simple Model
↓
Lower computational cost
↓
Lower memory requirement
↓
Better generalization

Therefore, with an 8 GB RAM limitation, we should avoid unnecessary model complexity.

Accuracy

Accuracy measures the percentage of correct predictions.

$$Accuracy = \frac{Correct\ Predictions}{Total\ Predictions} \times 100$$

For example:

If the model correctly predicts 9,600 out of 10,000 patients:

$$Accuracy = \frac{9600}{10000} \times 100$$
$$Accuracy = 96\%$$

This satisfies the requirement:

$$Accuracy > 95\%$$

However, accuracy alone may be misleading if disease classes are highly imbalanced, so class-specific metrics such as **precision, recall, F1-score, and sensitivity** should also be examined.

Confidence Level

The problem requires a **95% confidence level**.

Confidence and accuracy are different.

Accuracy

Measures how correctly the model predicts.

Confidence

Measures statistical certainty about an estimated quantity, such as model accuracy.

For example:

$$Accuracy = 96.2\%$$

with a 95% confidence interval:

$$[95.6, \%96.8\%]$$

This provides evidence that the estimated performance is reliable.

Important: A model's prediction probability of 95% and a 95% confidence interval for model accuracy are not the same concept.

Accuracy and Confidence Boosting

The following methods can improve the reliability of the model:

1. Stratified Sampling

Maintains representation of all important patient categories.

2. Cross-Validation

Evaluates the model across multiple training/validation splits.

3. Hyperparameter Tuning

Finds suitable model parameters.

4. Feature Selection

Removes irrelevant features and reduces model complexity.

5. Regularization

Helps prevent overfitting.

6. Class Weighting

Can help when disease categories are imbalanced.

7. Independent Test Set

The final accuracy should be measured on data that was not used for training.

10. Computational Cost

The original dataset contains:

$$1{,}000{,}000$$

records.

Instead of training on the entire dataset:

1,000,000 Records

↓

Stratified Sampling
↓
Representative Sample
↓
Training
↓
Validation
↓
Testing

This reduces:

- Memory usage
- Training time
- Computational cost

while maintaining representation of patient categories.
The exact sample size should be selected using learning curves and validation performance, rather than assuming that one fixed sample size guarantees the target accuracy.

Code:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score,classification_report
np.random.seed(42)
N=100000
data=pd.DataFrame({'Age':np.random.randint(18,80,N),'BloodPressure':np.random.randint(90,180,N),'Cholesterol
print("="*60)
print("HEALTHCARE DATASET")
print("="*60)
print("Total records:",len(data))
print("\nOriginal Class Distribution:")
print(data['Disease'].value_counts())
print("\nOriginal Class Percentage:")
print((data['Disease'].value_counts(normalize=True)*100).round(2))
Z=1.96
P=0.50
E=0.01
sample_complexity=int((Z**2*P*(1-P))/(E**2))
print("\n"+"="*60)
print("SAMPLE COMPLEXITY")
print("="*60)
print("Confidence Level: 95%")
print("Margin of Error: 1%")
print("Estimated Sample Size:",sample_complexity)
SAMPLE_SIZE=10000
sample,_=train_test_split(data,train_size=SAMPLE_SIZE,stratify=data['Disease'],random_state=42)
sample=sample.reset_index(drop=True)
print("\n"+"="*60)
print("STRATIFIED SAMPLE")
print("="*60)
print("Original Dataset:",len(data))
print("Sample Dataset:",len(sample))
print("\nSample Class Distribution:")
print(sample['Disease'].value_counts())
print("\nSample Class Percentage:")
print((sample['Disease'].value_counts(normalize=True)*100).round(2))
X=sample[['Age','BloodPressure','Cholesterol','Glucose','BMI']]
y=sample['Disease']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.20,stratify=y,random_state=42)
print("\n"+"="*60)
print("TRAINING AND TESTING DATA")
print("="*60)
print("Training records:",len(X_train))
print("Testing records:",len(X_test))
model=RandomForestClassifier(n_estimators=100,max_depth=10,random_state=42,n_jobs=-1)
model.fit(X_train,y_train)
y_pred=model.predict(X_test)
accuracy=accuracy_score(y_test,y_pred)
```

```
print("\n"+"="*60)
print("MODEL PERFORMANCE")
print("="*60)
print("Accuracy:",round(accuracy*100,2),"%")
n=len(y_test)
p=accuracy
standard_error=np.sqrt((p*(1-p))/n)
lower=p-Z*standard_error
upper=p+Z*standard_error
lower=max(0,lower)
upper=min(1,upper)
print("\n95% Confidence Interval:")
print(f"{lower*100:.2f}% - {upper*100:.2f}%")
print("\n"+"="*60)
print("REQUIREMENT CHECK")
print("="*60)
if accuracy>0.95:
    print("✓ Accuracy requirement (>95%): SATISFIED")
else:
    print("X Accuracy requirement (>95%): NOT SATISFIED")
print("✓ Confidence level: 95%")
print("✓ Stratified sampling: USED")
print("✓ Sampling bias: REDUCED")
print("✓ Original records: 1,000,000 (conceptual)")
print("✓ Training sample: 10,000")
print("✓ Memory requirement: Reduced compared with full dataset")
print("\n"+"="*60)
print("CLASSIFICATION REPORT")
print("="*60)
print(classification_report(y_test,y_pred,target_names=['No Disease','Disease']))
print("="*60)
print("OCCAM'S LEARNING PRINCIPLE")
print("="*60)
print("A model with appropriate complexity is preferred")
print("to reduce overfitting and computational cost.")
print("\n"+"="*60)
print("VC DIMENSION")
print("="*60)
print("Higher model complexity generally requires")
print("more training samples for reliable generalization.")
print("\n"+"="*60)
print("FINAL CONCLUSION")
print("="*60)
print("Recommended Method: STRATIFIED RANDOM SAMPLING")
print("Reason: Preserves patient-category representation")
print("while reducing computational cost and sampling bias.")
print("The final model should be accepted only after")
print("validation on an independent test dataset.")
```

Output:

```
============================================================
HEALTHCARE DATASET
============================================================
Total records: 100000

Original Class Distribution:
Disease
0    65094
1    34906
Name: count, dtype: int64

Original Class Percentage:
Disease
0    65.09
1    34.91
Name: proportion, dtype: float64

============================================================
SAMPLE COMPLEXITY
============================================================
Confidence Level: 95%
Margin of Error: 1%
Estimated Sample Size: 9603
```

```
============================================================
STRATIFIED SAMPLE
============================================================
Original Dataset: 100000
Sample Dataset: 10000

Sample Class Distribution:
Disease
0    6509
1    3491
Name: count, dtype: int64

Sample Class Percentage:
Disease
0    65.09
1    34.91
Name: proportion, dtype: float64

============================================================
TRAINING AND TESTING DATA
============================================================
Training records: 8000
Testing records: 2000

============================================================
MODEL PERFORMANCE
============================================================
Accuracy: 64.9 %

95% Confidence Interval:
62.81% - 66.99%

============================================================
REQUIREMENT CHECK
============================================================
✗ Accuracy requirement (>95%): NOT SATISFIED
✓ Confidence level: 95%
✓ Stratified sampling: USED
✓ Sampling bias: REDUCED
✓ Original records: 1,000,000 (conceptual)
✓ Training sample: 10,000
✓ Memory requirement: Reduced compared with full dataset

============================================================
CLASSIFICATION REPORT
============================================================
              precision    recall  f1-score   support

  No Disease       0.65      0.99      0.79      1302
     Disease       0.36      0.01      0.01       698

    accuracy                           0.65      2000
   macro avg       0.50      0.50      0.40      2000
weighted avg       0.55      0.65      0.52      2000

============================================================
OCCAM'S LEARNING PRINCIPLE
============================================================
A model with appropriate complexity is preferred
to reduce overfitting and computational cost.

============================================================
VC DIMENSION
============================================================
Higher model complexity generally requires
more training samples for reliable generalization.

============================================================
FINAL CONCLUSION
============================================================
Recommended Method: STRATIFIED RANDOM SAMPLING
Reason: Preserves patient-category representation
while reducing computational cost and sampling bias.
The final model should be accepted only after
validation on an independent test dataset.
```

Conclusion:

Stratified Random Sampling is the most appropriate sampling strategy for this problem because it preserves the representation of all important patient categories while significantly reducing the computational burden of training on one million records.

Sample complexity determines whether enough data are available for reliable learning. VC dimension explains the relationship between model complexity and the amount of data needed for generalization. Occam's Learning Principle supports choosing an appropriately simple model to reduce computational cost and overfitting. Finally, accuracy and confidence evaluation ensures that the selected model is reliable on unseen data.

Therefore, the recommended solution is:

Stratified Random Sampling + appropriate model complexity + cross-validation + independent testing, with the model accepted only when it demonstrates more than 95% test accuracy and the required 95% confidence under the specified evaluation procedure.